Study of Various Network Commands used in Linux & Window.

AIM: To Study the Various Network Commands used in Linux and Window.

Requirement Analysis:

* Desktop Computer
* Printer
* Command prompt.

Procedure:

Basic Networking Commands in Linux & windows

- arp - ARP is short form of address resolution protocol It will show the IP address of your computer along with the IP address and MAC address of your router
- hostname: This is the simplest of all TCP/IP Commands. It simply display the name of your computer
- ipConfig/all: This command displays detailed configuration information about your TCP/IP Connection, including Router, Gateway, DNS, DHCP, and type of Ethernet adapter in your system.
- nbtstat -a: This Command helps solve problems with NetBIOS name resolution.
- netstat: Netstat displays a variety of statistics about a Compelters active TCP/IP Connection.

- NS lookup: It is used to display DNS details, such as the IP address of a particular computer, the MX records for a domain or the NS servers of a domain

- Pathping: Pathping is unique to windows and is basically a combination of the Ping and Tracert commands.

- Ping: Ping use ICMPE to communicate to other dev

- Route: Route Command is used to show/manipulate the IP routing table.

Linux Commands:

- IP: The ip Command is one of the basic comma every administration will need in daily work, from setting up new system and assigning IPs to troublshooting existing system.

- If Config: The if Config Command was/is a staple in many Sysadmin's tools belt for configuration a troubleshooting networks

- mtr - MTR is a program with a command line interface that serves as a network diagnostic and troubleshooting tool

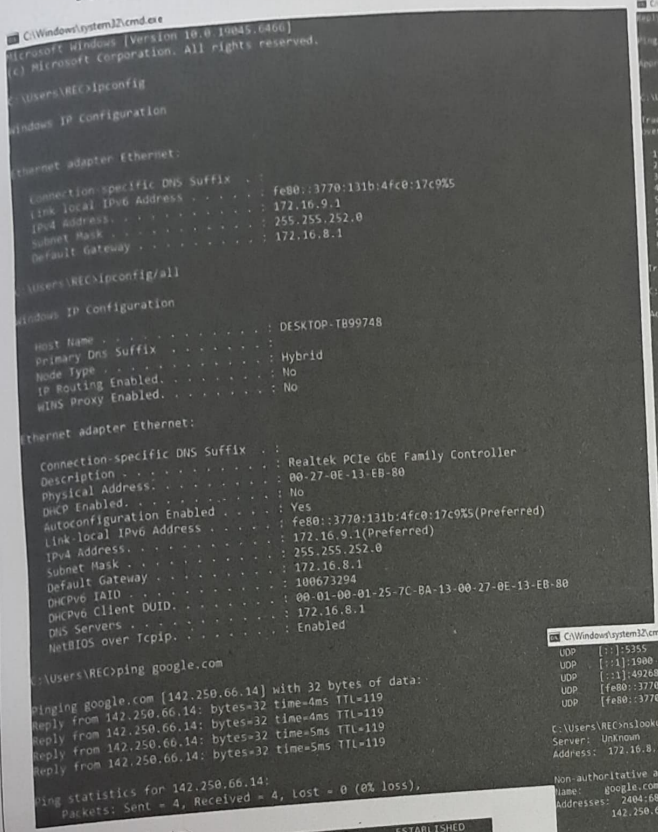
- TCPdump: The TCPdump command is designed for Capturing and displaying Packets.

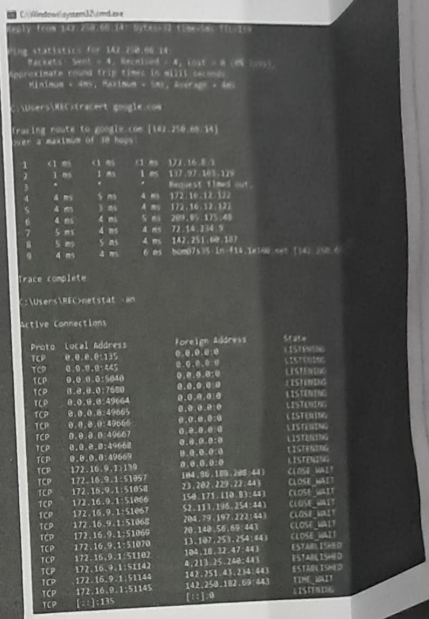
- Ping: Ping is a tool that verifies IP-level connectivity to another TCP/IP Computer by sending Internet Control Message protocol (ICMP) echo Request message.

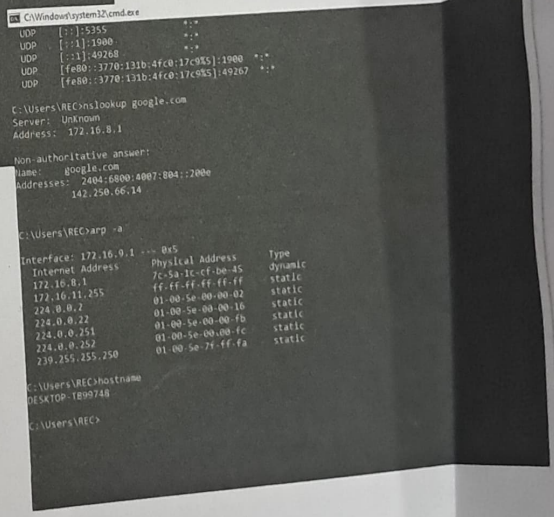
Exercise. No : 1

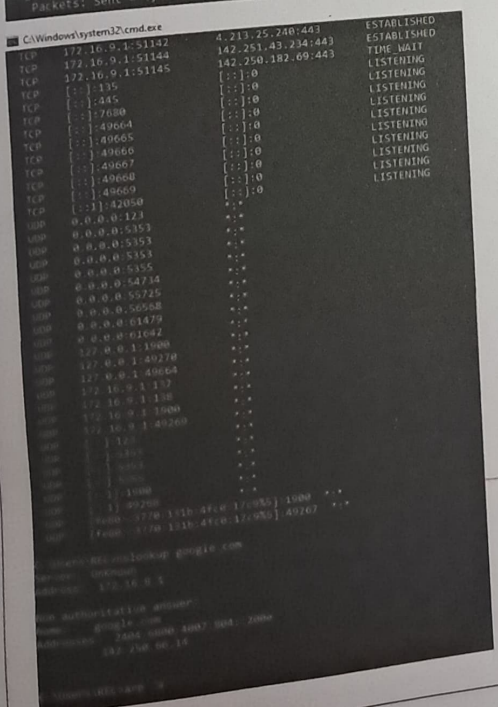
Basic Linux and Windows commands

Windows :









```
224.0.0.22            01-00-5e-00-00-16     static
224.0.0.251           01-00-5e-00-00-fb     static
224.0.0.252           01-00-5e-00-00-fc     static
239.255.255.250       01-00-5e-7f-ff-fa     static

C:\Users\REC>hostname
DESKTOP-TB09748

C:\Users\REC>pathping

Usage: pathping [-g host-list] [-h maximum_hops] [-i address] [-n]
                [-p period] [-q num_queries] [-w timeout]
                [-4] [-6] target_name

Options:
    -g host-list      Loose source route along host-list.
    -h maximum_hops   Maximum number of hops to search for target.
    -i address        Use the specified source address.
    -n                Do not resolve addresses to hostnames.
    -p period         Wait period milliseconds between pings.
    -q num_queries    Number of queries per hop.
    -w timeout        Wait timeout milliseconds for each reply.
    -4                Force using IPv4.
    -6                Force using IPv6.

C:\Users\REC>
```

```
Select C:\Windows\system32\cmd.exe

Interface List
  ...
IPv4 Route Table
  ...
IPv6 Route Table
  ...
```

C:\Windows\system32\cmd.exe

```
    -w timeout      Wait timeout milliseconds for each reply.
    -4              Force using IPv4.
    -6              Force using IPv6.

C:\Users\REC>route

Manipulates network routing tables.

ROUTE [-f] [-p] [-4|-6] command [destination]
                  [MASK netmask]  [gateway] [METRIC metric]  [IF interface]

    -f            Clears the routing tables of all gateway entries.  If this is
                  used in conjunction with one of the commands, the tables are
                  cleared prior to running the command.

    -p            When used with the ADD command, makes a route persistent across
                  boots of the system.  By default, routes are not preserved
                  when the system is restarted.  Ignored for all other commands,
                  which always affect the appropriate persistent routes.

    -4            Force using IPv4.

    -6            Force using IPv6.

    command       One of these:
                    PRINT     Prints  a route
                    ADD       Adds    a route
                    DELETE    Deletes a route
                    CHANGE    Modifies an existing route
    destination   Specifies the host.
    MASK          Specifies that the next parameter is the 'netmask' value.
    netmask       Specifies a subnet mask value for this route entry.
                  If not specified, it defaults to 255.255.255.255.
    gateway       Specifies gateway.
    interface     the interface number for the specified route.
    METRIC        specifies the metric, ie. cost for the destination.

All symbolic names used for destination are looked up in the network database
file NETWORKS. The symbolic names for gateway are looked up in the host name
database file HOSTS.

If the command is PRINT or DELETE. Destination or gateway can be a wildcard,
(wildcard is specified as a star '*'), or the gateway argument may be omitted.

If Dest contains a * or ?, it is treated as a shell pattern, and only
matching destination routes are printed. The '*' matches any string,
and '?' matches any one char. Examples: 157.*.1, 157.*, 127.*, *224*.

Pattern match is only allowed in PRINT command.
Diagnostic Notes:
    Invalid MASK generates an error, that is when (DEST & MASK) != DEST.
```

Linux :

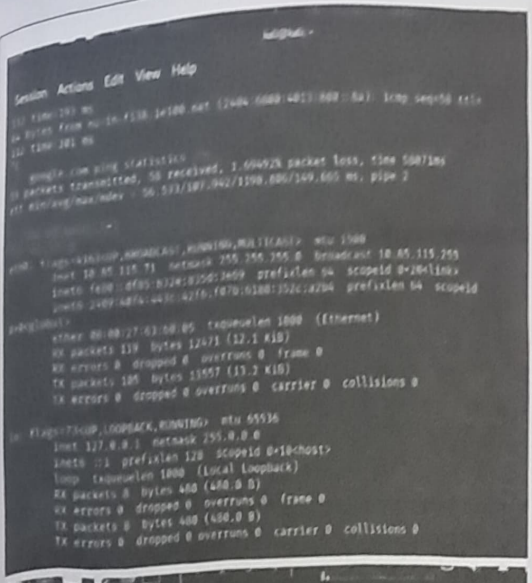
```
kali@kali: ~
Session  Actions  Edit  View  Help
zsh: corrupt history file /home/kali/.zsh_history
┌──(kali㉿kali)-[~]
└─$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def
ault qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
    link/ether 08:00:27:63:b0:05 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute
 eth0
       valid_lft 2752sec preferred_lft 2752sec
    inet6 2404:6800:4013:800:a27c:fe79:6188:322c:a2b4/64 scope global dynamic nop
refixroute
       valid_lft 6974sec preferred_lft 6974sec
    inet6 fe80::a00:27ff:fe63:b005/64 scope link noprefixroute
       valid_lft forever preferred_lft forever

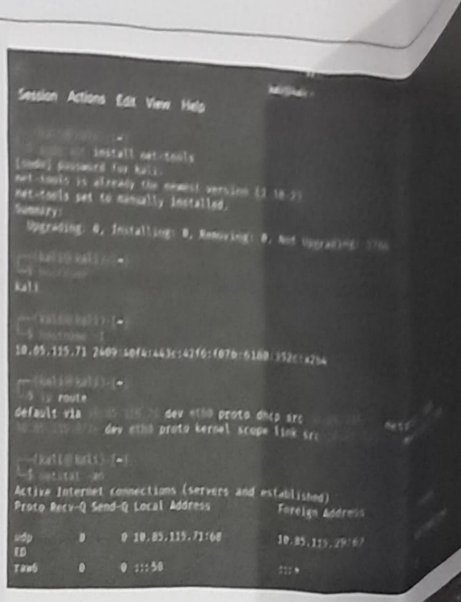
┌──(kali㉿kali)-[~]
└─$ ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFA
ULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
```

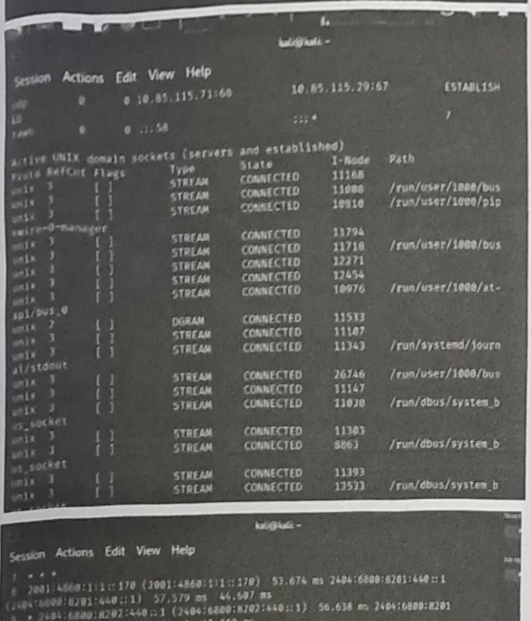
```
kali@kali: ~
Session  Actions  Edit  View  Help

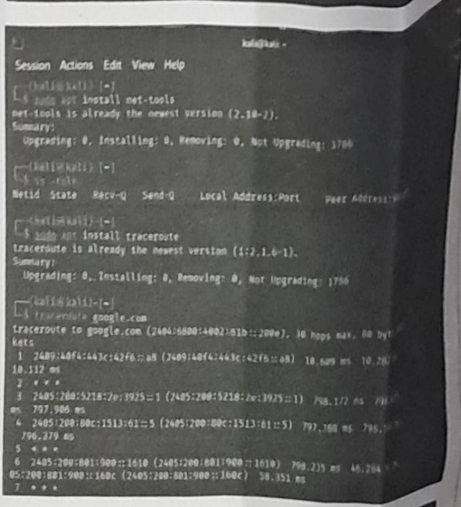
┌──(kali㉿kali)-[~]
└─$ ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFA
ULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup DEFAULT group default qlen 1000
    link/ether 08:00:27:63:b0:05 brd ff:ff:ff:ff:ff:ff

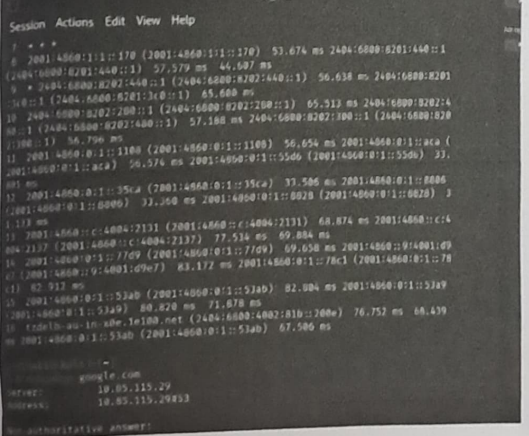
┌──(kali㉿kali)-[~]
└─$ ping google.com
PING google.com (2404:6800:4013:800::8a) 56 data bytes
64 bytes from nu-in-f138.1e100.net (2404:6800:4013:800::8a): icmp_seq=1 ttl=
12 time=1199 ms
64 bytes from nu-in-f138.1e100.net (2404:6800:4013:800::8a): icmp_seq=2 ttl=
12 time=174 ms
64 bytes from nu-in-f138.1e100.net (2404:6800:4013:800::8a): icmp_seq=3 ttl=
12 time=68.7 ms
64 bytes from nu-in-f138.1e100.net (2404:6800:4013:800::8a): icmp_seq=4 ttl=
12 time=72.0 ms
64 bytes from nu-in-f138.1e100.net (2404:6800:4013:800::8a): icmp_seq=5 ttl=
12 time=67.9 ms
64 bytes from nu-in-f138.1e100.net (2404:6800:4013:800::8a): icmp_seq=6 ttl=
12 time=73.0 ms
64 bytes from nu-in-f138.1e100.net (2404:6800:4013:800::8a): icmp_seq=7 ttl=
12 time=60.6 ms
64 bytes from nu-in-f138.1e100.net (2404:6800:4013:800::8a): icmp_seq=8 ttl=
12 time=108 ms
64 bytes from nu-in-f138.1e100.net (2404:6800:4013:800::8a): icmp_seq=9 ttl=
12 time=84.0 ms
```

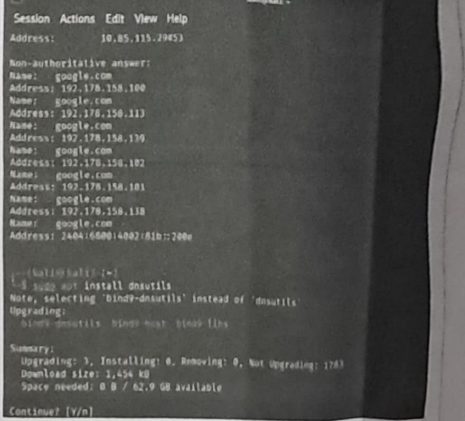

kali@kali: ~
Session Actions Edit View Help
icmp_seq=50 ttl=
time=191 ms
bytes from no-in-x38.1e100.net (2404:6800:4011:80f::8a): icmp_seq=50 ttl=
time=201 ms
google.com ping statistics
packets transmitted, 50 received, 1.66492% packet loss, time 50071ms
rtt min/avg/max/mdev = 56.533/107.942/1398.686/149.665 ms, pipe 2
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
inet 10.85.115.71 netmask 255.255.255.0 broadcast 10.85.115.255
inet6 fe80::d6b5:b72e:8350:3e69 prefixlen 64 scopeid 0x20<link>
inet6 2409:40f4:443c:42f6:f07b:6180:352c:a2b4 prefixlen 64 scopeid
ether 00:0c:29:63:60:05 txqueuelen 1000 (Ethernet)
RX packets 119 bytes 12471 (12.1 KiB)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 105 bytes 13557 (13.2 KiB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
inet 127.0.0.1 netmask 255.0.0.0
inet6 ::1 prefixlen 128 scopeid 0x10<host>
loop txqueuelen 1000 (Local Loopback)
RX packets 8 bytes 480 (480.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 8 bytes 480 (480.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

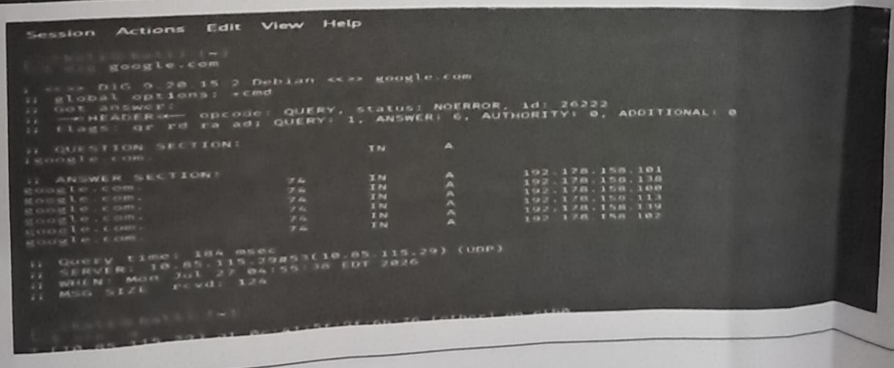

kali@kali: ~
Session Actions Edit View Help
install net-tools
[sudo] password for kali:
net-tools is already the newest version (2.10-1).
net-tools set to manually installed.
Summary:
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1766
kali
10.85.115.71 2409:40f4:443c:42f6:f07b:6180:352c:a2b4
route
default via dev eth0 proto dhcp src
dev eth0 proto kernel scope link src
netstat -an
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address Foreign Address
udp 0 0 10.85.115.71:68 10.85.115.29:67
ED
raw6 0 0 :::58 :::*


kali@kali: ~
Session Actions Edit View Help
udp 0 0 10.85.115.71:68 10.85.115.29:67 ESTABLISH
LD :::* 7
raw6 0 0 :::58 :::*
Active UNIX domain sockets (servers and established)
Proto RefCnt Flags Type State I-Node Path
unix 3 [ ] STREAM CONNECTED 11168
unix 3 [ ] STREAM CONNECTED 11080 /run/user/1000/bus
unix 3 [ ] STREAM CONNECTED 10810 /run/user/1000/pip
ewire-0-manager
unix 3 [ ] STREAM CONNECTED 11794
unix 3 [ ] STREAM CONNECTED 11710 /run/user/1000/bus
unix 3 [ ] STREAM CONNECTED 12271
unix 3 [ ] STREAM CONNECTED 12454
unix 3 [ ] STREAM CONNECTED 10976 /run/user/1000/at-
spi/bus_0
unix 2 [ ] DGRAM CONNECTED 11513
unix 3 [ ] STREAM CONNECTED 11107
unix 3 [ ] STREAM CONNECTED 11343 /run/systemd/journ
al/stdout
unix 3 [ ] STREAM CONNECTED 26746 /run/user/1000/bus
unix 3 [ ] STREAM CONNECTED 11147
unix 3 [ ] STREAM CONNECTED 11010 /run/dbus/system_b
us_socket
unix 3 [ ] STREAM CONNECTED 11303
unix 3 [ ] STREAM CONNECTED 8063 /run/dbus/system_b
us_socket
unix 3 [ ] STREAM CONNECTED 11393
unix 3 [ ] STREAM CONNECTED 13533 /run/dbus/system_b


kali@kali: ~
Session Actions Edit View Help
sudo apt install net-tools
net-tools is already the newest version (2.10-1).
Summary:
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1766
ss -nxlp
Netid State Recv-Q Send-Q Local Address:Port Peer Address:P
sudo apt install traceroute
traceroute is already the newest version (1:2.1.6-1).
Summary:
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1766
traceroute google.com
traceroute to google.com (2404:6800:4002:81b::200e), 30 hops max, 60 byt
kets
1 2409:40f4:443c:42f6::a8 (2409:40f4:443c:42f6::a8) 10.609 ms 10.282
10.112 ms
2 * * *
3 2405:200:5218:2e:3925::1 (2405:200:5218:2e:3925::1) 798.172 ms 798.1
ms 797.986 ms
4 2405:200:80c:1513:61::5 (2405:200:80c:1513:61::5) 797.760 ms 796.3
796.379 ms
5 * * *
6 2405:200:801:900::1610 (2405:200:801:900::1610) 798.225 ms 46.284
05:200:801:900::160c (2405:200:801:900::160c) 58.351 ms
7 * * *


kali@kali: ~
Session Actions Edit View Help
7 * * *
8 2001:4860:1:1::170 (2001:4860:1:1::170) 53.674 ms 2404:6800:8201:440::1
(2404:6800:8201:440::1) 57.579 ms 44.607 ms
9 * 2404:6800:8202:440::1 (2404:6800:8202:440::1) 56.638 ms 2404:6800:8201
:3c0::1 (2404:6800:8201:3c0::1) 65.600 ms
10 2404:6800:8202:200::1 (2404:6800:8202:200::1) 65.513 ms 2404:6800:8202:4
2:100::1) 56.796 ms
11 2001:4860:0:1::1108 (2001:4860:0:1::1108) 56.654 ms 2001:4860:0:1::aca (
2001:4860:0:1::aca) 56.576 ms 2001:4860:0:1::55d6 (2001:4860:0:1::55d6) 33.
695 ms
12 2001:4860:0:1::35ca (2001:4860:0:1::35ca) 33.506 ms 2001:4860:0:1::8806
(2001:4860:0:1::8806) 33.360 ms 2001:4860:0:1::8028 (2001:4860:0:1::8028) 3
1.173 ms
13 2001:4860::c:4804:2131 (2001:4860::c:4804:2131) 68.874 ms 2001:4860::c:4
804:2137 (2001:4860::c:4804:2137) 77.534 ms 69.884 ms
14 2001:4860:0:1::77d9 (2001:4860:0:1::77d9) 69.658 ms 2001:4860::9:4001:d9
15 2001:4860:0:1::78c1 (2001:4860:0:1::78c1) 83.172 ms 2001:4860:0:1::77d9
(2001:4860::9:4001:d9e7) 83.172 ms
16 82.917 ms
17 2001:4860:0:1::53a9 (2001:4860:0:1::53a9) 82.804 ms 2001:4860:0:1::53a9
(2001:4860:0:1::53a9) 80.820 ms 71.878 ms
18 tzdelb-au-in-x0e.1e100.net (2404:6800:4002:81b::200e) 76.752 ms 68.439
ms 2001:4860:0:1::53ab (2001:4860:0:1::53ab) 67.506 ms
google.com
Server: 10.85.115.29
Address: 10.85.115.29#53
Non-authoritative answer:


kali@kali: ~
Session Actions Edit View Help
Address: 10.85.115.29#53
Non-authoritative answer:
Name: google.com
Address: 192.178.158.100
Name: google.com
Address: 192.178.150.113
Name: google.com
Address: 192.178.158.139
Name: google.com
Address: 192.178.158.102
Name: google.com
Address: 192.178.158.101
Name: google.com
Address: 192.178.150.138
Name: google.com
Address: 2404:6800:4002:81b::200e
sudo apt install dnsutils
Note, selecting 'bind9-dnsutils' instead of 'dnsutils'
Upgrading:
bind9-dnsutils bind9-host bind9-libs
Summary:
Upgrading: 3, Installing: 0, Removing: 0, Not Upgrading: 1763
Download size: 1,454 kB
Space needed: 0 B / 62.9 GB available
Continue? [Y/n]


Session Actions Edit View Help
google.com
<<>> DiG 9.20.15-2-Debian <<>> google.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 26222
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 6, AUTHORITY: 0, ADDITIONAL: 0
;; QUESTION SECTION:
;google.com. IN A
;; ANSWER SECTION:
google.com. 74 IN A 192.178.158.101
google.com. 74 IN A 192.178.150.138
google.com. 74 IN A 192.178.158.100
google.com. 74 IN A 192.178.150.113
google.com. 74 IN A 192.178.158.139
google.com. 74 IN A 192.178.158.102
;; Query time: 184 msec
;; SERVER: 10.85.115.29#53(10.85.115.29) (UDP)
;; WHEN: Mon Jul 27 09:55:38 EDT 2026
;; MSG SIZE rcvd: 124

The steps to assigning the ip address Manually.

AIM: To assign the steps to assigning the ip address manually

Requirement Analysis:

- Desktop Computer
- Printer
- HDD/SSD

Procedure:

- Turn on the Computer and log in with an administrator account.
- Press window + R to open the Run dialog box
- Type and press enter to open network Connections
- Identify the active network adapter
- Righ-click the network adapter and selection properties
- In the Properties Window select Internet Protocol Version 4 (TCP/IPv4)
- click the properties button
- Select use the following Ip address
- Enter the Subnet Mask
- Enter the Default Gateway
- Select use the following DNS server address
- Enter the preffered DNS server
- Click OK. then click close to save the settings.
- open Cmd prompt, run ipconfig and verify that the manually assigned Ip address is displayed correctly.

Input:

Computer 1 IP address : 192.168.1.1
Computer 2 IP address : 192.168.1.2
Subnet mask : 255.255.255.0
Default Gateway : 192.168.1.254
DNS sever : 8.8.8.8

Output:

The IP address were successfully assigned manually to the computers and connectivity b/w Computer was successfully verified using the ping Command

27/11/25

classmate
Date
Page

Exp-3.-Study of Different Types of Network Cables and Crimping of Cable with RJ45 Connector.

AIM: To study the different type of network cables and crimp an ethernet cable using a RJ45 connector

Types of Network Cables.

1) Twisted Pair cables
   - UTP (Unshielded Twisted pair)
   - STP (Shielded Twisted pair)

2) Coaxial Cable.

3) optical fibre cable.

RJ45 color code - T568B (Default order)

Pin	Color
1	white-orange
2	orange
3	white green
4	Blue
5	white blue
6	Green
7	white brown
8	Brown

Order:

White orange → orange → white green → Blue → white blue → Green → white brown → Brown

Input:

1) Cat 5e / Cat 6 Ethernet Cable.
2) RJ45 Connector
3) Crimping tool
4) Cable Tester

Color sequence input

$W_O - O - W_G - B - W_B - G - W_{Br} - Br$

Output:

The network cable was successfully crim
with an RJ45 Connector using the T568B
Color Standard and the cable connection wa
Verified using a cable tester.

Ex no: 4

Study of installation and user interface of Cisco Packet tracked.

AIM:

To study the installation and user interface of Cisco ~~Networking~~ tracker and understand its environment.

Installation:

1.) Download Cisco Packet tracker from Cisco Networking Academy.
2.) Login and download the suitable version
3.) Run the installer

PROCEDURE:

1.) Place 4 PCs and a Hub in the workspace.
2.) place another 4 PCs and a switch.
3.) Connect PC using Copper Straight-through cables.
4.) Assign IP address and subnet mask to PC
5.) Use PDU tool to send a Packet from one PC to another
6.) Observe the flow in simulation mode
7.) Repeat the process for PC's connected to the switch.
8.) Observe and compare the packet-forwarding behaviour.

Output:

Thus, Cisco Packet tracer was studied successfully and behavior of hub and switch in a simple network was observed.

Ex 5 Design and Configure a Simple network using one router and 2 switches.

AIM:

To design and configure a simple network using one router, two switches and four PCs in Cisco Packet tracer.

PROCEDURE:

1) Create the topology with 4PC, 2 switches (2960) and 1 router.
2) Connect PC0 and PC1 to Switch 0
3) Connect PC_2 and PC_3 to Switch 1
4) Connect both switch to the router using copper straight-Through wires
5) Configure the PCs with Static IP address
6) Configure router interface.
   → Gig0/0/0 → 192.168.10.1/24
   → Gig0/0/1 → 192.168.20.1/24
7) Enable both interface using no shutdown command.
8) Verify connectivity using Ping command.

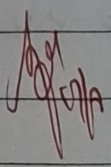
OBServation:

→ Both LANS were successfully connected through router
→ PCs within some LAN communicated successfully
→ Successfully Ping replies confirmed network connectivity.

Assignment
Date
Page

Output:

The network topology was successfully designed and configured in cisco packet tracer b/w both LAN was verified.



Exno 6 Setting up a Local Area Network

AIM:

To design and Configure a LAN connecting four PCs through a switch, assign static IP addresses, verify connectivity.

Procedure:

1) Open Cisco Packet tracer
2) Place 4 PCs and switch (2760) in the workspace
3) Connect all PCs to the switch using copper straight through Cables.
4) Configure static IP addresses for all PCs
5) Configure the switch management IP through the CLI

Commands:

- enable
- Configure terminal
- interface Vlan 1
- ip address 10.1.1.5 255.0.0.0
- no shutdown
- exit

6) Open the Command Prompt of a PC and use the Ping command to verify connectivity.
7) Add a server and Configure its IP and HTTP service to demonstrate data sharing
8) Switch to simulation mode and observe the packet flow through the switch

Result:

- The LAN was successfully designed and configured using Cisco Packet Tracer.

9/10/2025

classmate
Date
Page

Output:

The LAN was successfully designed and configure using Cisco Packet Tracer.

18/8/26

Exp no: 7 Customize switch with network modules using Cisco packet tracer

AIM:

To Customize a modular switch by installing network modules and configure a fiber optic connection b/w switch and router

Procedure:

1) Open Cisco Packet tracer and place - PT-Empty, two PCs and a Cisco 2911 router
2) Open the switch physical tab and Power it off
3) Install the required modules
   - -> PT-Switch-NM-1FFE => slot 0 and slot 1
   - -> PT-Switch-NM-1FGbE => slot 2
4) Power ON the switch and open the CLI
5) Verify the Installed interfaces.

   Command:

   enable.

   show ip interface brief

6) Connect

   PC0 → Fast Ethernet 0/1 using copper cable

   PC1 → Fast Ethernet 1/1 using copper cable

   Switch Gigabit Ethernet 2/1 Router using fiber cable
7) Power OFF the Cisco 2911 router and install the HWIC-1GE-SFP
8) Power ON the router and connect the fibre cable to the newly added Gigabit interface

9) Configure the router interface:
   - enable
   - Configure terminal
   - interface Gigabit Ethernet 0/0/0
   - ip address 192.168.10.1 255.255.0

Ex no: 7 Customize switch with network modules using Cisco Packet tracker.

AIM:

To Customize a modular switch by installing network modules and configure a fiber-optic connection b/w switch and router.

Procedure:

1.) Open Cisco Packet tracer and place -PT-Empty, two PCs and a Cisco 2911 router.
2.) Open the switch physical tab and power it off
3.) Install the required modules.
   -> PT-Switch-NM-1FFE => slot 0 and slot 1
   -> PT-Switch-NM-1FGE => slot 2
4.) Power ON the switch and open the CLI
5.) Verify the installed interfaces.
   Command:
   enable.
   show ip interface brief.
6.) Connect
   PC0 -> Fast Ethernet 0/1 using copper cable
   PC1 -> Fast Ethernet 1/1 using copper cable.
   Switch Gigabit Ethernet 2/1 Router using fiber cable
7.) Power OFF the Cisco 2911 router and install the HWIC-1GE-SFP
8.) Power ON the router and connect the fibre Cable to the newly added Gigabit interface.

9.) Configure the router interface:
- enable
- Configure terminal
- interface Gigabit Ethernet 0/0/0
- ip address 192.168.10.1 255.255.0

- no shut down
- exit
- exit
- Write memory.

10) Verify the interface:

show ip interface brief.

11) Test Connectivity using:

Ping 192.168.10.2.

Output:

The modular switch was successfully customized with copper and fiber interface and a fiber optic connection b/w switch and router was configured successfully.

classmate
Date
Page

Exp-8 - Hamming Code

AIM:

To write a program to implement error detection and correction using the Hamming code concepts and to verify the error correction feature with a test run.

Procedure:

1.) Sender: Read the text and convert each character to 8-bit binary.

2.) for 8 data bits, 4 parity bits are needed ($2^r \geq m+r+1$, $8 \Rightarrow 4$) the code word is 12 bits.

3.) Write the 12-bits code words into the file channel.txt

4.) Receiver: Read channel.txt and take 12 bits at a time

5.) Recalculate the parity checks; the parity-check bits from the error position.

6.) if the value 0 - no error, otherwise display the error position and flip that bit to correct it.

7.) Remove the parity bits, join the 8 data bits, convert to ASCII character and display the message.

Program:

sender.py

```
text = input("Enter text: ")
def encode(bits):
    n = 12
    code = [0]*(n+1)
    j = 0
    for i in range(1, n+1):
        if i & (i-1):
            code[i] = bits[j]; j += 1
    for p in (1,2,4,8)
    code[p] = sum(code[i] for i in range(1,n+1) if i&p and i!=p)%2
    return ''.join(map(str, code[1:]))
```

```
out = ''
for ch in text:
    bits = [int(b) for b in format(ord(ch), '08b')]
    out += encode(bits)
open("channel.txt", 'w').write(out)
print("Binary data:", ' '.join(format(ord(c), '08b')
print("Hamming code:", out)
```

receiver.py

```
data = open("channel.txt").read().strip()
msg = ''
for k in range(0, len(data), 12):
    code = [0] + [int(b) for b in data[k:k+12]]
    pos = 0
    for p in (1,2,4,8):
        if sum(code[i] for i in range(1,13) if i & p) % 2:
            pos += p
    if pos
        print(f"Block {k//12+1}: error at position
                                        {pos} -> corr
        code[pos] ^= 1
    bits = ''.join(str(code[i]) for i in range(1,13) if i &
    msg += chr(int(bits,2))
print("Decoded message:", msg)
```

Input:

- Run sender: Hi

output:

sender

Enter text: Hi

Binary listing: 01001000 01101001

Hamming Code: 000110000110 001111000101

Receiver Out:

Decoded message: Hi

Receiver Run:

Block 1: Error at position 6 → Corrected

Decoded message: Hi

Result:

Thus the Hamming code program was implemented the single bit error was detected its position displayed, corrected and the original message was recovered.